תקציר שיעור 1

יעילות: למה סופרים צעדים?

תוכנית נכונה זה לא מספיק

שתי תוכניות נכונות יכולות להיות שונות מאוד במהירות. כדי לדעת את זה מראש, סופרים **צעדים** ושואלים איך מספרם גדל כש-n גדל.

לזכור

1. **נכון זה לא מספיק.** שתי תוכניות נכונות יכולות להיות שונות מאוד במהירות על קלט גדול.
2. **סופרים צעדים, לא שניות.** צעדים לא תלויים במחשב; הם גדלים באותו קצב כמו הזמן.
3. **קצב גדילה, מקרה גרוע.** שואלים: n מוכפל — הצעדים נשארים, מוכפלים או פי 4? תמיד עבור המקרה הגרוע.

דוגמאות מהשיעור

ספירת צעדים: 1 ו-n

```
static int First(int[] arr)
{
    steps++;
    return arr[0];
}

static int Sum(int[] arr)
{
    int sum = 0;
    for (int i = 0; i < arr.Length; i++)
    {
        steps++;
        sum += arr[i];
    }
    return sum;
}
```

- `First` עושה תמיד צעד `1`, לא משנה מה גודל המערך.
- `Sum` עושה `n` צעדים: אחד לכל תא.
- עבור `n = 10, 100, 1000` יוצאים `10, 100, 1000` צעדים: כש-n מוכפל, הצעדים מוכפלים.

ספירת צעדים: כל הזוגות

```
static int CountPairs(int[] arr)
{
    int count = 0;
    for (int i = 0; i < arr.Length; i++)
    {
        for (int j = i + 1; j < arr.Length; j++)
        {
            steps++;
            if (arr[i] == arr[j])
                count++;
        }
    }
    return count;
}
```

- עבור `i = 0` הלולאה הפנימית רצה `n-1` פעמים, עבור `i = 1` היא רצה `n-2` פעמים, וכן הלאה עד 0.
- סך הכול `n(n-1)/2` צעדים — בערך $n^2/2$.
- עבור `n = 10, 100, 1000` יוצאים `45, 4950, 499500` צעדים: כש-n גדל פי 10, הצעדים גדלים פי 100.

כמה מהר הם גדלים?

n	First (1)	Sum (n)	CountPairs (≈ n²/2)
10	1	10	45
100	1	100	4,950
1,000	1	1,000	499,500
1,000,000	1	1,000,000	≈ 500,000,000,000

שימו לב

! זמן בשניות תלוי במחשב ומשתנה מהרצה להרצה. מספר הצעדים לא משתנה.

! שאלת המפתח: כש-n מוכפל, הצעדים נשארים כמו שהם, מוכפלים או גדלים פי 4?

! תמיד מנתחים את **המקרה הגרוע**. למשל `IndexOf` כשהאיבר לא נמצא בכלל: n צעדים.

! לולאה פנימית שמתחילה ב-`j = i + 1` עושה `n(n−1)/2` צעדים, בערך $n^2/2$.